

Web-Scraping and Automated Data Collection

Rebecca Kittel, Post-Doctoral Researcher
Free University Berlin & WZB
contact: rebecca.kittel@fu-berlin.de

21 July 2026

Studied in Cologne



PhD in Florence, European University Institute



Research on Populism and Communication Strategies



e.g. Let's talk populist? A survey experiment on effects of (non-) populist discourse on vote choice or Simply Speaking? Language Complexity among (Non-)Populist Actors in Parliamentary Debates

Protest Camps in Authoritarian Regimes



see [LOOPS Project](#)

Who are you?

What do you already know about web-scraping? And is there any website from which you may need data?



Goals for Today

- ▶ Understanding basic HTML structures
- ▶ Scraping individual elements from a webpage
- ▶ Setting up a workflow to scrape static webpages

Outlook of Today

- ▶ **First Block:** Introduction to web-scraping and understanding the logic of webpages
- ▶ **Coffee Break:** 15:00h to 15.15h
- ▶ **Second Block:** Different web-scraping methods to acquire your data from webpages → Example: Press Releases from Extinction Rebellion UK

Important! Please download the course materials:

<https://tinyurl.com/3nna3bhu>

Or use your new GitHub skills:

https://github.com/kittelr/webscraping_mannheim

Introduction to Web scraping with R



Introduction to Web scraping with R

- ▶ Web scraping is a very useful tool for extracting data from web pages
 - ▶ various types of websites can be scraped, from university webpages to social media
 - ▶ lots of different techniques
- ▶ Today: We will focus on scraping simple static pages!
 - ▶ extracting text, links and tables from many standard webpages
 - ▶ Conditions: page written in HTML/XML, page has a static URL through which we can reach it
- ▶ Procedure
 - ▶ 'parsing' page in R
 - ▶ extracting relevant parts
 - ▶ cleaning into usable format (e.g. data frame, raw text...)

HTML - Hyper Text Markup Language

- ▶ HTML pages display a formatting style that we can use to select specific elements of that page and retrieve the information embedded in these formatting structure

```
<> first_example.html X
C: > Users > rkitt > Documents > PostDoc > year_2 > women_summer_school_bremen >
1  | <!DOCTYPE html>
2  | <html>
3  | <body>
4  |
5  | <h1>My First Heading</h1>
6  | <p>My first paragraph.</p>
7  |
8  | </body>
9  | </html> |
```

My First Heading

My first paragraph.

HTML - Hyper Text Markup Language

markup: additional description of formatting beyond the content of the text

- ▶ **tags**: distinguish types of content
 - ▶ e.g., headings (h1, h2, etc.), paragraphs (p), tables (table), lists (ul,ol), images (img) → they 'surround' the actual text content
- ▶ **attributes**: specify characteristics of elements; most importantly...
 - ▶ "class" → manipulate the appearance and group web elements (references in .css)
 - ▶ "id" → allow to uniquely identify (e.g. email-input or password-input)
 - ▶ "src" (for *img*) or "href" (for *a* (link)) → relative links to other web content (images or pages)

```
▼ <a class="s-content-card--link" href="/our-work/news/us-states-cities-
and-businesses-will-continue-lead-climate-action"> flex
  ▼ <div class="s-content-card__title-content"> flex
    ▼ <h4 class="s-content-card__title">
      ▼ <span>
        US states, cities, and businesses will continue to lead on
        climate action
      </span>
    </h4>
  </div class="s-content-card__info"> ... </div> flex
```

Basic HTML

- ▶ **Body:** This is where we are mostly interested at → content of a webpage is stored within the body structure
- ▶ **Head:** This provides meta information → often used by search engines, sometimes we use these information too, e.g. if we want to know when the webpages was published and we don't find it in the body structure
- ▶ HTML documents consists of web elements that are organized hierarchically → HTML tree, elements can be nested within each other which can help to locate them
- ▶ Parent elements have children (child element)
- ▶ Elements (can) have preceding/following siblings

```
<html>
  <head>
    <title>Title of your
web page</title>
  </head>
  <body>
    HTML web page content
  </body>
</html>
```

HTML - Tables

```
<table>
  <thead>
    <tr>
      <th>Capital</th>
      <th>Country</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>Rome</td>
      <td>Italy</td>
    </tr>
    <tr>
      <td>Paris</td>
      <td>France</td>
    </tr>
    <tr>
      <td>Madrid</td>
      <td>Spain</td>
    </tr>
  </tbody>
</table>
```

Capital	Country
Rome	Italy
Paris	France
Madrid	Spain

HTML - Building Blocks

1. Attributes, additional information about an element

- ▶ Examples:

- ▶ `<div attribute="x"> Title </div>`

- ▶ ``

2. Links, text turned into a link by using an `<a>` tag with a "href" attribute (hyperreference)

- ▶ This is text `<a href="http://quotes.toscrape.com/">with a link</a>`. → This is text with a link.

3. Styling, determines how they appear in the Browser

- ▶ CSS file: `p.success{color: green}`

- ▶ HTML file: `<p class="success">Success!</p>` → Success!

Simple/Static Pages

- ▶ `rvest` → `r` packages for scraping
 - ▶ Strengths: covers most frequent use cases, integration with other packages, e.g. `tidyverse`
 - ▶ Weaknesses: relatively simple: no dynamic webpages
 - ▶ Main uses: Tables, texts, extracting links, (downloading files)
- ▶ Link to R for Data Science: <https://r4ds.hadley.nz/webscraping>

'Parsing' HTML

- ▶ Manually specify a resource
- ▶ R sends a request to server that hosts website
- ▶ Server returns resource
- ▶ R parses HTML (i.e interprets the structure), but does not render it in a nice fashion
- ▶ Tell R which parts of the structure to focus on and what to extract

Hands-On HTML Parsing

→ Open R-Script: 01_basic_rvest.R

The process of Scraping

```
url <- "https://www.presidency.ucsb.edu/" Specify URL  
page <- read_html(url) ,parse' URL
```

The Process of Scraping

```
url <- "https://www.presidency.ucsb.edu/" Specify URL  
page <- read_html(url) ,parse' URL  
sel_nodes <- html_nodes(page) Select parts
```

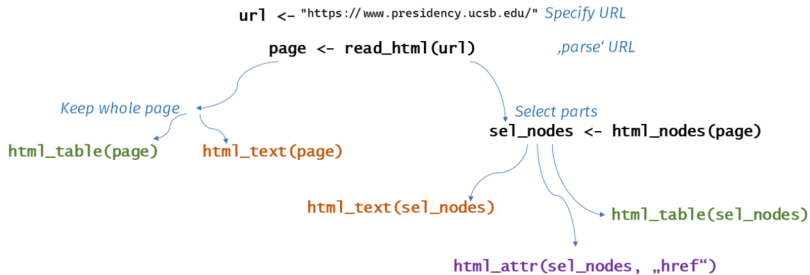
Keep whole page ←

The Process of Scraping

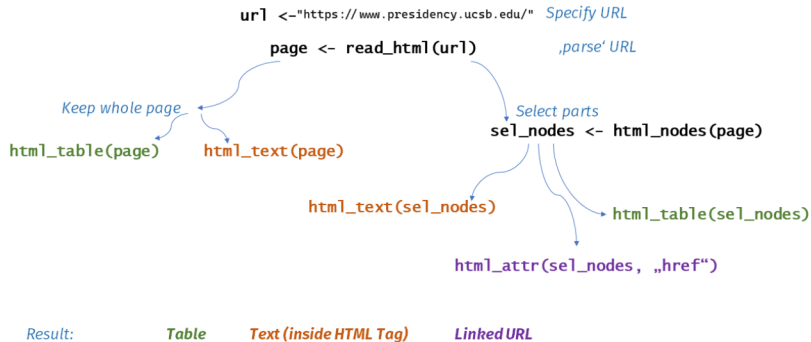
```
url <- "https://www.presidency.ucsb.edu/" Specify URL  
page <- read_html(url) ,parse' URL  
sel_nodes <- html_nodes(page) Select parts
```

Keep whole page ←

The Process of Scraping



The Process of Scraping



CSS (Cascading Style Sheet) Selectors

- ▶ we use the appearance / style of text to select specific parts
- ▶ based on specific **HTML elements**: tags, classes, attributes
- ▶ CSS selectors provide a *language* in which we can describe what we select at a more abstract level

Basic selectors

*	universal selector	Matches everything.	*
element	element / type selector	Matches an element	p
[attribute]	attribute selector	Matches elements containing a given attribute	[href]
[attribute=value]	attribute selector	Matches elements containing a given attribute with a given value	[href=/]
.class	class selector	Matches the value of a class attribute	.header
#id	ID selector	Matches the value of an id attribute	#first

More complex attribute selectors

<code>[attribute*=value]</code>	Matches elements with an attribute that contains a given value	<code>a[href*="pressrelease"]</code>
<code>[attribute^="value"]</code>	Matches elements with an attribute that starts with a given value	<code>a[href^="/press/"]</code>
<code>[attribute\$="value"]</code>	Matches elements with an attribute that ends with a given value	<code>[href\$=".pdf"]</code>

Combining CSS Selectors

There are several ways to combine CSS Selectors:

element,element	Selects all <div> elements and all <p> elements	div, p
element element	Selects all <p> elements inside <div> elements	div p
element>element	Selects all <p> elements where the parent is a <div> element	div > p
element+element	Selects all <p> elements that are placed immediately after <div> elements	div + p
element1~element2	Selects every element that are preceded by a <p> element	p ~ ul

Selectorgadget

Selectorgadget is an open source tool that makes [CSS selector](#) generation and discovery on complicated sites a breeze. Just install the [Chrome Extension](#) or drag the bookmarklet to your bookmark bar, then go to any page and launch it. A box will open in the bottom right of the website. Click on a page element that you would like your selector to match (it will turn green). Selectorgadget will then generate a minimal CSS selector for that element, and will highlight (yellow) everything that is matched by the selector. Now click on a highlighted element to remove it from the selector (red), or click on an unhighlighted element to add it to the selector. Through this process of selection and rejection, Selectorgadget helps you come up with the perfect CSS selector for your needs.

How could I use this?

- for webpage scraping with tools such as [Nokogiri](#) and [Beautiful Soup](#)
- to generate [jQuery](#) selectors for dynamic sites
- as a tool to examine JavaScript-generated DOM structures
- as a tool to help you style only particular elements on the page with your stylesheets
- for selenium or phantomjs testing

Created by [Andrew Cantino](#) and Kyle Maxwell. You can find the [current version on GitHub](#), and please feel free to leave comments below.

.block

Clear (3)

Toggle Position

XPath

Help

X

Useful Links:

- ▶ Selectorgadget: <https://selectorgadget.com/> or <https://rvest.tidyverse.org/articles/selectorgadget.html>
- ▶ Vignette: <https://rvest.tidyverse.org/articles/selectorgadget.html>

Refine by Name

Donald J. Trump (1st Term)

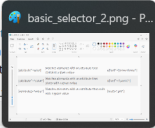
Submit

Category

- Campaign Documents
- Convention Speeches
- Debates
- Determinations

RESULTS 451 - 475 of 979 records found

Date	Related	Document Title
Feb 19, 2019	Donald J. Trump (1st Term)	Memorandum on Establishment of the United States Space Force deny our use of space in a crisis or conflict. It is imperative ...
Feb 26, 2019	Donald J. Trump (1st Term)	Statement of Administration Policy: H.J. Res. 46 - Terminating the National Emergency Declared by the President on the Southern Border humanitarian and security crisis that threatens core national ... effectively to the ongoing crisis at the Southern Border. The ...
Feb 26, 2019	Donald J. Trump (1st Term)	Fact Sheet - The Funds Available to Address the National Emergency at Our Border security and humanitarian crisis that threatens core national ...
Feb 27, 2019	Donald J. Trump (1st Term)	White House Press Release - Challenge for New Solutions unprecedented public health addition crisis . This competi that is devastating too many
Mar 02, 2019	Donald J. Trump (1st Term)	...text-nowrap , .text-nowrap a



basic_selector_2.png - P...nches \$1.55M

... addressing the

ress the opioid **crisis**

ear (79) Toggle Position XPath Help X

Hands-On Scraping Multiple URLs

→ Open R-Script: 02_webscraping_multiple_sites.R

Coffee Break :)

Please find your partner!

Game Time!

Please click on this link:

<https://flukeout.github.io/#>

Press Releases as Data Source - Example



Press Releases as Data Source in Political Science, e.g. Gessler and Hunger. 2022. How the refugee crisis and radical right parties shape party competition on immigration or Erfort et al. 2023. The PARTYPRESS Database: A new comparative database of parties' press releases

Pair-Programming

- ▶ Learning technique for programming
- ▶ Essentials:
 - ▶ Get into pairs
 - ▶ Choose one computer
 - ▶ One person will be the *Driver* → Types into the computer
 - ▶ One person will be the *Navigator* → Navigates the code and explains what to write (still talk to each other, like navigating a car)
 - ▶ Every 5 min switch roles

Hands-On Scraping Multiple URLs

→ Open R-Script: 03__press__releases__example.R

Ethical Considerations and Academic Debate



Levels of Regulation

- ▶ **laws:** criminal code, copyright laws etc.
- ▶ **terms of service:** by platform that specify conditions of use
- ▶ **ethical regulations:** (privacy, GDPR, informed consent) → can be discussed with university internal review boards (applying for ethical approval , providing a data management plan)
- ▶ **technical measures:** they can prevent scraping, especially in place on big websites

→ Web-scraping is often considered a grey area, most websites do not like to be scraped but legal consequences are often not clear; however, researchers should be aware of possible consequences and their responsibility, especially when scraping individual level data (human subjects compliance, privacy laws, ethical responsibility)

Outlook: What else is out there?

- ▶ APIs (e.g. [Telegram](#), [abgeordnetenwatch.de](#) or [TikTok](#)) → However, many APIs pose restrictions on researcher, can only download specific data or you have to pay these days (e.g. X, former Twitter API)
- ▶ Dynamic Websites and dynamic webpage scraping as a possible way around

Short Introduction to Dynamic Websites

- ▶ many webpages are *interactive*: they provide more content as we interact, e.g. scroll or click...
- ▶ automatic reveal of more articles based on scrolling or load more buttons
- ▶ more difficult to scrape as the URL does not change

Short Introduction to Dynamic Websites

- ▶ RSelenium, based on Selenium → However, currently many troubles in setting it up as phantomjs a headless browser is deprecated
- ▶ Playwright (open-source automation library) is a new and powerful tool that can also tackle newer websites → However, can only be used in python, recommended with pycharm
- ▶ Useful Links:
 1. [Installing Python](#)
 2. [Setting-up Pycharm](#)
 3. [Playwright for Python](#)
 4. [Playwright with Pycharm](#)

Playwright Example – Python

```
1 import re
2 from playwright.sync_api import Playwright, sync_playwright, expect
3
4
5 def run(playwright: Playwright) -> None:
6     browser = playwright.chromium.launch(headless=False)
7     context = browser.new_context()
8     page = context.new_page()
9     page.goto("https://www.amazon.de/")
10    page.get_by_role("button", name="Akzeptieren").click()
11    page.get_by_role("searchbox", name="Amazon.de durchsuchen").click()
12    page.get_by_role("searchbox", name="Amazon.de durchsuchen").fill("brille")
13    page.get_by_role("button", name="Los").click()
14    page.get_by_role("searchbox", name="Amazon.de durchsuchen").click()
15    page.get_by_role("button", name="Menü Alle Kategorien öffnen").click()
16    page.get_by_role("button", name="Menü schließen").click()
17    page.get_by_role("link", name="Angebote", exact=True).click()
18    page.get_by_role("searchbox", name="Amazon.de durchsuchen").click()
19    page.get_by_role("searchbox", name="Amazon.de durchsuchen").fill("brille")
20    page.get_by_role("searchbox", name="Amazon.de durchsuchen").press("Enter")
21
22    # -----
23    context.close()
24    browser.close()
25
26
27 with sync_playwright() as playwright:
28     run(playwright)
29
```

Your Research Ideas?

- ▶ Do you have any ideas or cases for which you are planning or now considering using web-scraping methods?

Your Research Ideas?

- ▶ Do you have any ideas or cases for which you are planning or now considering using web-scraping methods?

Goals for Today – Achieved? And any other feedback you may have!

- ▶ Understanding basic HTML structures
- ▶ Scraping individual elements from a webpage
- ▶ Setting up a workflow to scrape static webpages

Please answer here:



Thank you and hope to see you somewhere again :)